Numerical Programming

CP2: Estimating mass, velocity and drag force using ODEs

Demetre Lataria

2024-12-25

Chapter 1

Introduction

This document explains the Python implementation of a simulation of a ball's motion using the Runge-Kutta 4th order method (RK4). The motion is described by an ordinary differential equation (ODE) system, accounting for drag forces and collisions with the boundaries of a rectangular container. We analyze the governing equations, parameters, numerical method, and its advantages and disadvantages.
The model describes ball, or more accurately a disk which we are looking from top and which is sliding on some surface with rigid walls for disk to bounce off of.
I use my object detection and velocity approximation code from CP1 and image scaling from AP3. Thus i won't be explaining the part of code there since it was already done on previous presentations. The only thing changed in CP1's original code is skipping preprocessing of the frame since the demo videos I provide are simple enough and give the most accurate results with object detection and velocity approximation was modified to get immediate velocity to lessen the error from average of velocities not updating fast enough to generate valid trajectory.
I have also written the generator of simulations which has it's own real values printed at the end so you can check for any combination of parameters of initial velocity, drag force and mass.

Chapter 2

The functions used

The motion of the ball is governed by Newton's second law of motion:

$$\vec{F} = m\vec{a}$$

where $\vec{F}$ is the net force acting on the ball, m is its mass, and $\vec{a}$ is the acceleration. The forces considered in the simulation include:

- **Drag Force** ($\vec{F}_d$): Opposes the motion of the ball and is proportional to the square of the velocity. It is given by:

  $$\vec{F}_d = \tfrac{1}{2} C_d \rho A \|\vec{v}\|^2 \hat{v}$$

  where C_d is the drag coefficient which is constant and is 0.5, ρ is the air density, $A = \pi r^2$ is the cross-sectional area of the ball, and $\hat{v}$ is the unit vector in the direction of velocity.

- **Collision Force**: When the ball collides with the boundaries of the container, its velocity is reversed, and the restitution coefficient e is applied to model energy loss. However in our case restitution is not estimated and is always 1, which can be changed if restitution of ball in video is known. Since restitution is property of material and surface roughness of the ball these can not be estimated simply by using object detection.

The ODE system to be solved is:

$$\frac{d\vec{r}}{dt} = \vec{v}$$

$$\frac{d\vec{v}}{dt} = \vec{a} = \left(-\frac{C_d}{m} \vec{v} \right)$$

where $\vec{r}$ is the position vector and $\vec{v}$ is the velocity vector.

Chapter 3

Parameters

The simulation uses the following parameters:

- r: Radius of the ball.
- e: Coefficient of restitution ($0 < e \leq 1$) (again, always 1 in our case.)
- $\vec{\textbf{pos}}$: Initial position vector $[x, y]$.
- $\vec{\textbf{vel}}$: Initial velocity vector $[v_x, v_y]$.
- $\vec{\textbf{accel}}$: Initial acceleration vector $[a_x, a_y]$.
- dt: Time step for numerical integration. (Equals to the frametime of the source video.)
- W, H: Width and height of the container. (Equals to the resolution of the video.)
- $t_{\textbf{max}}$: Total simulation time. (Equals to the length of the video.)

Chapter 4

Selected method for calculating ODE

The RK4 method is a numerical technique for solving ODEs. It provides a higher accuracy compared to simpler methods like Euler's method. For an ODE of the form:

$$\frac{dy}{dt} = f(t, y)$$

the RK4 method computes the next value of y as:

$$y_{n+1} = y_n + \frac{dt}{6}\left(k_1 + 2k_2 + 2k_3 + k_4\right)$$

where:

$$k_1 = f(t_n, y_n)$$
$$k_2 = f\left(t_n + \frac{dt}{2}, y_n + \frac{dt}{2}k_1\right)$$
$$k_3 = f\left(t_n + \frac{dt}{2}, y_n + \frac{dt}{2}k_2\right)$$
$$k_4 = f\left(t_n + dt, y_n + dt \cdot k_3\right)$$

The RK4 method is used to integrate the ODEs for the ball's position and velocity. At each step, the method computes intermediate slopes (k_1, k_2, k_3, k_4) for both position and velocity, and updates their values accordingly. Collisions with the container boundaries are handled by reversing the velocity components and applying the restitution coefficient.

Chapter 5

Encountered issues

There are a lot of issues with the implementation of such estimating program. First of all, Since we are working with pixel information from the video and object detection works with pixels as well, We can observe that when ball is moving in the simulation, the resolution is finite and it might move so little that pixels either won't get updated or they jump from one extreme to another. Thus each derivation of offset, velocity, acceleration and so on, get more and more inaccurate with bigger jumps. For this reason in my code you will notice that mass can not be determined in wanted accuracy and end estimations can not determin it at all, since it is calculated from drag force and acceleration and acceleration is too unstable and inacurrate to use. This might be solved with interpolation of positions but this introduces latency and might not be enough with so much aliasing and pixels switching from one extreme to another. It will also require bigger domain, more resolution to get the optimal amount of data to work with. This can be solved with object detection code which is weighted towards anti-aliased pixels and can determin how much an object has moved from the intensity of the pixel instead of just checking if its 0 (black) or 1 (white).
It is worth noting that velocity and drag force themselves are estimated well enough and can be used for simple videos.

Chapter 6

Provided videos

I have provided three videos with this project. One of them uses gravity which will not work with this model, since it describes 2d disk on surface and not a 2d ball bouncing around.

6.1 ball_simulation_0

Initial position set to [2.0, 1.0], initial velocity set to $[1.7, -0.3]$:

```
Real:
    mass: 1
    velocity: 1.4543677894343525
    drag: 0.05437997214577085

Last Estimated:
    mass: -1
    velocity: 1.5728549056023255
    drag: 0.05355324734744906
```

6.2 ball_simulation_1

Initial velocity set to $[3.0, 0.0]$

```
Real:
    mass: 5
    velocity: 2.8170171689109678
    drag: 0.18298283108903018
```

```
Last Estimated:
    mass: -1
    velocity: 2.916666666666666
    drag: 0.18415439358567118
```

6.3 ball_simulation_gravity_fail

Same as last one but with gravity. Drag estimation fails significantly.

```
Real:
    mass: 5
    velocity: 9.505335246505181
    drag: 0.8829370822201793

    Last Estimated:
        mass: -1
        velocity: 9.470493092064475
        drag: 1.9415727652138344
```

Chapter 7

Conclusion

As I discussed before, weighted object detection or way more resolution can fix this issue.